

```
In [4]: !pip install torch torchvision transformers opencv-python pillow
```

```
Requirement already satisfied: torch in c:\users\karth\anaconda3\lib\site-packages (2.6.0+cpu)
Requirement already satisfied: torchvision in c:\users\karth\anaconda3\lib\site-packages (0.21.0+cpu)
Requirement already satisfied: transformers in c:\users\karth\anaconda3\lib\site-packages (4.51.1)
Requirement already satisfied: opencv-python in c:\users\karth\appdata\roaming\python\python312\site-packages (4.11.0.86)
Requirement already satisfied: pillow in c:\users\karth\anaconda3\lib\site-packages (10.3.0)
Requirement already satisfied: filelock in c:\users\karth\anaconda3\lib\site-packages (from torch) (3.13.1)
Requirement already satisfied: typing-extensions>=4.10.0 in c:\users\karth\anaconda3\lib\site-packages (from torch) (4.11.0)
Requirement already satisfied: networkx in c:\users\karth\anaconda3\lib\site-packages (from torch) (3.2.1)
Requirement already satisfied: jinja2 in c:\users\karth\anaconda3\lib\site-packages (from torch) (3.1.4)
Requirement already satisfied: fsspec in c:\users\karth\anaconda3\lib\site-packages (from torch) (2024.3.1)
Requirement already satisfied: setuptools in c:\users\karth\anaconda3\lib\site-packages (from torch) (69.5.1)
Requirement already satisfied: sympy==1.13.1 in c:\users\karth\anaconda3\lib\site-packages (from torch) (1.13.1)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in c:\users\karth\anaconda3\lib\site-packages (from sympy==1.13.1->torch) (1.3.0)
Requirement already satisfied: numpy in c:\users\karth\anaconda3\lib\site-packages (from torchvision) (1.26.4)
Requirement already satisfied: huggingface-hub<1.0,>=0.30.0 in c:\users\karth\anaconda3\lib\site-packages (from transformers) (0.30.2)
Requirement already satisfied: packaging>=20.0 in c:\users\karth\anaconda3\lib\site-packages (from transformers) (23.2)
Requirement already satisfied: pyyaml>=5.1 in c:\users\karth\anaconda3\lib\site-packages (from transformers) (6.0.1)
Requirement already satisfied: regex!=2019.12.17 in c:\users\karth\anaconda3\lib\site-packages (from transformers) (2023.10.3)
Requirement already satisfied: requests in c:\users\karth\anaconda3\lib\site-packages (from transformers) (2.32.2)
Requirement already satisfied: tokenizers<0.22,>=0.21 in c:\users\karth\anaconda3\lib\site-packages (from transformers) (0.21.1)
Requirement already satisfied: safetensors>=0.4.3 in c:\users\karth\anaconda3\lib\site-packages (from transformers) (0.5.3)
Requirement already satisfied: tqdm>=4.27 in c:\users\karth\anaconda3\lib\site-packages (from transformers) (4.66.4)
Requirement already satisfied: colorama in c:\users\karth\anaconda3\lib\site-packages (from tqdm>=4.27->transformers) (0.4.6)
Requirement already satisfied: MarkupSafe>=2.0 in c:\users\karth\anaconda3\lib\site-packages (from jinja2->torch) (2.1.3)
Requirement already satisfied: charset-normalizer<4,>=2 in c:\users\karth\anaconda3\lib\site-packages (from requests->transformers) (2.0.4)
Requirement already satisfied: idna<4,>=2.5 in c:\users\karth\anaconda3\lib\site-packages (from requests->transformers) (3.7)
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\karth\anaconda3\lib\site-packages (from requests->transformers) (2.2.2)
Requirement already satisfied: certifi>=2017.4.17 in c:\users\karth\anaconda3\lib\site-packages (from requests->transformers) (2025.1.31)
```

```

In [21]: import torch
from transformers import CLIPProcessor, CLIPModel
import cv2
from PIL import Image
import numpy as np

# Custom waste categories
waste_classes = [
    "a pile of non-recyclable trash like used tissue paper, dirty food wrappers,
    "clean white or colored office paper sheets, magazines, or books ready for r
    "clear or green glass bottles or jars, like wine bottles, jam jars, or broke
    "fresh organic waste including vegetable peels, fruit skins, banana peels, a
    "crumpled plastic water bottles, plastic containers, shopping bags, and clin
    "discarded electronic items such as smartphones, earphones, USB cables, and
    "shiny aluminum soda cans, beer cans, and tin food containers",
    "flattened brown cardboard boxes, shipping cartons, or cereal boxes",
    "used AA batteries, coin cells, or lithium-ion batteries from old electronic
    "fast food containers made of styrofoam, greasy pizza boxes, and plastic cut
    "a pile of used textiles or clothing waste like old shirts, socks, and torn
    "a cluster of biodegradable food waste including bread crusts, egg shells, a
    "broken ceramic mugs, shattered plates, and porcelain tiles",
    "medical waste like disposable masks, gloves, used bandages, and medicine bl
    "construction debris like bricks, concrete chunks, wood splinters, and nails
    "garden waste such as leaves, twigs, branches, and grass clippings",
    "hazardous chemical containers like pesticide bottles or bleach cans with wa
    "a tangled bunch of wires, circuit boards, and motherboard scraps from broke
    "old CDs, DVDs, floppy disks, and scratched discs with labels",
    "ink cartridges, toner boxes, and printer ribbons from office electronic was
]

# Load CLIP model + processor
model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32")
processor = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")

device = "cuda" if torch.cuda.is_available() else "cpu"
model.to(device)

# Start webcam
cap = cv2.VideoCapture(0)
if not cap.isOpened():
    print("Error: Cannot access webcam.")
    exit()

print("Running... Press 'q' to quit.")

while True:
    ret, frame = cap.read()
    if not ret:
        print("Failed to grab frame.")
        break

    img_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    pil_img = Image.fromarray(img_rgb)

    # Predict
    inputs = processor(text=waste_classes, images=pil_img, return_tensors="pt",
    outputs = model(**inputs)
    logits = outputs.logits_per_image

```

```

probs = logits.softmax(dim=1).cpu().detach().numpy()[0]
label = waste_classes[np.argmax(probs)]
conf = probs[np.argmax(probs)]

# Display on frame
cv2.putText(frame, f"{label} ({conf:.2f})", (10, 40),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)

cv2.imshow("Waste Detection", frame)
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()

```

Running... Press 'q' to quit.

```

In [31]: import torch
from transformers import CLIPProcessor, CLIPModel
import cv2
from PIL import Image
import numpy as np
from collections import deque, Counter

waste_classes = [
    # Plastics
    "a plastic water bottle, either clear or colored, with or without a label",
    "a used plastic food container, like a lunchbox or takeaway box",
    "a disposable plastic spoon, fork, or knife from fast food",
    "plastic shopping bags crumpled or torn",
    "a used plastic pen or marker with ink stains",
    "bubble wrap or plastic packaging film from delivery parcels",
    "a broken plastic toy or part of a plastic appliance",
    "plastic straws, lids, or disposable cups from a coffee shop",

    # Metals
    "a shiny aluminum soda can, crushed or whole",
    "a steel food can like those for beans or soup",
    "metal screws, nails, bolts, or hinges",
    "an old metal pan or cooking pot",
    "a metallic mobile phone body or aluminum casing",
    "a bundle of copper wires or coiled cables",

    # Paper & Cardboard
    "a crumpled sheet of notebook paper or printed paper",
    "a clean newspaper or magazine with visible pages",
    "a flattened cardboard shipping box with tape",
    "a paper cup or bowl stained with food or coffee",

    # Glass
    "a clear glass bottle for soda, water, or juice",
    "a green or brown beer bottle made of glass",
    "a broken glass jar with visible cracks or fragments",
    "a glass window shard or clear sheet of glass",

    # Food & Organic Waste
    "banana peels, apple cores, and other fruit scraps",
    "vegetable peels like potato, carrot, or onion skins",
    "leftover cooked food like rice, noodles, or meat pieces",
    "bread crusts, sandwich remains, or pastry leftovers",

```

```
"egg shells, tea bags, or coffee grounds",
"a rotting piece of fruit or moldy bread",

# Wood Waste
"wood shavings, splinters, or sawdust from carpentry",
"broken wooden furniture parts like chair legs or tabletops",
"tree branches, logs, or twigs from yard waste",

# E-Waste
"a broken mobile phone with visible screen cracks",
"a tangled bunch of USB cables or charging cords",
"a computer keyboard or mouse that no longer works",
"a printed circuit board with chips and resistors",
"an old television remote with missing buttons",

# Textiles
"a pile of old clothes like shirts, pants, or jackets",
"a torn piece of fabric or used bedsheet",
"a worn-out shoe or sandal with visible damage",

# Hazardous & Misc
"used batteries like AA, AAA, or coin cells",
"a bleach or pesticide plastic container with warning labels",
"a disposable surgical mask, gloves, or PPE kit",
"a medicine blister pack with pills",
"a used syringe or medical vial (for visual use only!)",
"ash, soot, or charcoal remnants from burned waste",
"dusty construction debris like bricks, rubble, or drywall pieces"
"all types of gum bottles, fevicol"

# Plastics
"a plastic water bottle, either clear or colored, with or without a label",
"a used plastic food container, like a lunchbox or takeaway box",
"a disposable plastic spoon, fork, or knife from fast food",
"plastic shopping bags crumpled or torn",
"a used plastic pen or marker with ink stains",
"bubble wrap or plastic packaging film from delivery parcels",
"a broken plastic toy or part of a plastic appliance",
"plastic straws, lids, or disposable cups from a coffee shop",
"a shampoo or detergent plastic bottle",
"a plastic comb or hairbrush",
"a used plastic file or folder from office supplies",
"a plastic flower pot or plant tray",

# Metals
"a shiny aluminum soda can, crushed or whole",
"a steel food can like those for beans or soup",
"metal screws, nails, bolts, or hinges",
"an old metal pan or cooking pot",
"a metallic mobile phone body or aluminum casing",
"a bundle of copper wires or coiled cables",
"a rusted metal pipe or rod",
"an empty deodorant or spray can with a metal top",
"a metal door handle or latch",
"a tin lid or metal bottle cap",

# Paper & Cardboard
"a crumpled sheet of notebook paper or printed paper",
"a clean newspaper or magazine with visible pages",
"a flattened cardboard shipping box with tape",
"a paper cup or bowl stained with food or coffee",
```

```
"a used tissue or paper towel",
"a torn cereal box or paperboard packaging",
"an old book with worn pages",
"a printed receipt or bus ticket",

# Glass
"a clear glass bottle for soda, water, or juice",
"a green or brown beer bottle made of glass",
"a broken glass jar with visible cracks or fragments",
"a glass window shard or clear sheet of glass",
"a broken mirror fragment",
"a laboratory glass test tube or beaker",
"a glass perfume bottle or container",

# Food & Organic Waste
"banana peels, apple cores, and other fruit scraps",
"vegetable peels like potato, carrot, or onion skins",
"leftover cooked food like rice, noodles, or meat pieces",
"bread crusts, sandwich remains, or pastry leftovers",
"egg shells, tea bags, or coffee grounds",
"a rotting piece of fruit or moldy bread",
"used coconut shells or sugarcane waste",
"wilted flowers or garden plant remains",

# Wood Waste
"wood shavings, splinters, or sawdust from carpentry",
"broken wooden furniture parts like chair legs or tabletops",
"tree branches, logs, or twigs from yard waste",
"a wooden crate or pallet",
"a discarded plywood board or MDF panel",
"a broken broomstick or wooden handle",

# E-Waste
"a broken mobile phone with visible screen cracks",
"a tangled bunch of USB cables or charging cords",
"a computer keyboard or mouse that no longer works",
"a printed circuit board with chips and resistors",
"an old television remote with missing buttons",
"a broken earphone or headphone",
"a discarded laptop charger or power adapter",
"a burnt-out LED bulb or CFL tube",

# Textiles
"a pile of old clothes like shirts, pants, or jackets",
"a torn piece of fabric or used bedsheet",
"a worn-out shoe or sandal with visible damage",
"a ripped pillow or cushion cover",
"a used towel or bathrobe",
"a discarded face mask made of cloth",

# Hazardous & Misc
"used batteries like AA, AAA, or coin cells",
"a bleach or pesticide plastic container with warning labels",
"a disposable surgical mask, gloves, or PPE kit",
"a medicine blister pack with pills",
"a used syringe or medical vial (for visual use only!)",
"ash, soot, or charcoal remnants from burned waste",
"dusty construction debris like bricks, rubble, or drywall pieces",
"all types of gum bottles, fevicol",
"a broken thermometer with mercury residue",
```

```
"a paint can with dried paint residue",  
"a cracked ceramic plate or cup"  
# Plastics  
"a plastic razor with a detachable blade head",  
"a plastic CD or DVD case",  
"a used plastic glue stick casing",  
"a plastic disposable lighter",  
  
# Metals  
"a broken wristwatch with a metal strap",  
"a rusted metal hinge or bracket from old furniture",  
"a scrap aluminum foil ball or sheet",  
"a metal key or padlock",  
  
# Paper & Cardboard  
"a used paper envelope with torn edges",  
"a shredded bank statement or confidential paper",  
"a greasy pizza box with oil stains",  
"a used paper placemat or fast-food wrapper",  
  
# Glass  
"a broken glass lightbulb",  
"a decorative glass piece or figurine",  
"a cracked car windshield glass",  
"a broken test tube from a science lab",  
  
# Food & Organic Waste  
"fish bones or meat bones from a meal",  
"corn husks or cobs after eating",  
"used lemon or citrus peels",  
"stale chapati or paratha pieces",  
  
# Wood Waste  
"a wooden popsicle stick",  
"a burnt matchstick",  
"wooden ice cream spoon",  
"an old wooden picture frame",  
  
# E-Waste  
"a damaged smartwatch or fitness band",  
"a broken hard disk or SSD",  
"a cracked tablet screen",  
"a battery pack from an old gadget",  
  
# Textiles  
"an old torn sock",  
"a discarded scarf or muffler",  
"a damaged curtain piece",  
"a worn-out cap or hat",  
  
# Hazardous & Misc  
"an expired spray paint can",  
"a broken compact fluorescent lamp (CFL)",  
"a dried nail polish or polish remover bottle",  
"a used mosquito coil or repellent patch",  
"an empty firecracker shell",  
"used wax strips or depilatory waste"
```

]

```

# Load model and processor
model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32")
processor = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")

device = "cuda" if torch.cuda.is_available() else "cpu"
model.to(device)

# Webcam
cap = cv2.VideoCapture(0)
if not cap.isOpened():
    print("Error: Webcam not accessible.")
    exit()

# Temporal smoothing buffer
last_preds = deque(maxlen=5)

print("Live waste classification running... Press 'q' to quit.")

while True:
    ret, frame = cap.read()
    if not ret:
        print("Error grabbing frame.")
        break

    # Convert to RGB and center-crop
    img_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    h, w, _ = img_rgb.shape
    min_dim = min(h, w)
    start_x = (w - min_dim) // 2
    start_y = (h - min_dim) // 2
    img_crop = img_rgb[start_y:start_y+min_dim, start_x:start_x+min_dim]
    pil_img = Image.fromarray(img_crop)

    # Prepare input
    inputs = processor(text=waste_classes, images=pil_img, return_tensors="pt",

    with torch.no_grad():
        outputs = model(**inputs)
        logits = outputs.logits_per_image
        probs = logits.softmax(dim=1).cpu().numpy()[0]

    top3_indices = np.argsort(probs)[-3:][::-1]
    top_label = waste_classes[top3_indices[0]]
    top_conf = probs[top3_indices[0]]

    # Add to smoothing buffer
    last_preds.append(top_label)
    smoothed_label = Counter(last_preds).most_common(1)[0][0]

    # Display only if confidence is high enough
    if top_conf > 0.50:
        display_label = f"{smoothed_label} ({top_conf:.2f})"
    else:
        display_label = "Unsure"

    # Draw on frame
    cv2.putText(frame, display_label, (10, 40),
                cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2)

    # Optional: Print top 3 predictions

```

```
print("\nTop 3 Predictions:")
for idx in top3_indices:
    print(f"{waste_classes[idx]}: {probs[idx]:.2f}")

# Show frame
cv2.imshow("Waste Detection", frame)
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

# Cleanup
cap.release()
cv2.destroyAllWindows()
```


Live waste classification running... Press 'q' to quit.

Top 3 Predictions:

stale chapati or paratha pieces: 0.16
a plastic comb or hairbrush: 0.12
a broken earphone or headphone: 0.06

Top 3 Predictions:

stale chapati or paratha pieces: 0.23
a broken earphone or headphone: 0.07
a shampoo or detergent plastic bottle: 0.06

Top 3 Predictions:

stale chapati or paratha pieces: 0.15
expired or spoiled dairy like cheese or paneer bits: 0.08
a shampoo or detergent plastic bottle: 0.06

Top 3 Predictions:

a torn piece of fabric or used bedsheet: 0.14
a torn piece of fabric or used bedsheet: 0.14
expired or spoiled dairy like cheese or paneer bits: 0.13

Top 3 Predictions:

stale chapati or paratha pieces: 0.14
expired or spoiled dairy like cheese or paneer bits: 0.11
a torn piece of fabric or used bedsheet: 0.09

Top 3 Predictions:

a broken mobile phone with visible screen cracks: 0.41
a broken mobile phone with visible screen cracks: 0.41
a metallic mobile phone body or aluminum casing: 0.06

Top 3 Predictions:

a broken mobile phone with visible screen cracks: 0.34
a broken mobile phone with visible screen cracks: 0.34
a metallic mobile phone body or aluminum casing: 0.06

Top 3 Predictions:

a broken mobile phone with visible screen cracks: 0.20
a broken mobile phone with visible screen cracks: 0.20
a metallic mobile phone body or aluminum casing: 0.19

Top 3 Predictions:

a metallic mobile phone body or aluminum casing: 0.25
a metallic mobile phone body or aluminum casing: 0.25
a battery pack from an old gadget: 0.13

Top 3 Predictions:

a metallic mobile phone body or aluminum casing: 0.29
a metallic mobile phone body or aluminum casing: 0.29
a broken mobile phone with visible screen cracks: 0.18

Top 3 Predictions:

a metallic mobile phone body or aluminum casing: 0.21
a metallic mobile phone body or aluminum casing: 0.21
a broken mobile phone with visible screen cracks: 0.16

Top 3 Predictions:

a piece of firewood with soot: 0.41
a plastic comb or hairbrush: 0.09

a burnt matchstick: 0.08

Top 3 Predictions:

stale chapati or paratha pieces: 0.18

expired or spoiled dairy like cheese or paneer bits: 0.09

a torn piece of fabric or used bedsheet: 0.07

Top 3 Predictions:

stale chapati or paratha pieces: 0.12

a faded cloth face mask with straps loose: 0.10

expired or spoiled dairy like cheese or paneer bits: 0.07

Top 3 Predictions:

a faded cloth face mask with straps loose: 0.17

expired or spoiled dairy like cheese or paneer bits: 0.09

a torn piece of fabric or used bedsheet: 0.06

```
In [6]: import torch
from transformers import CLIPProcessor, CLIPModel
import cv2
from PIL import Image
import numpy as np
from collections import deque, Counter

waste_classes = [
    #  PLASTIC WASTE
    "a crushed plastic water bottle with a blue cap and label",
    "a transparent plastic food container with leftover curry stains",
    "a disposable plastic spoon from a street food vendor",
    "a black plastic grocery bag caught in a tree",
    "a broken red plastic bucket with a missing handle",
    "a used plastic shampoo bottle with residue inside",
    "a squeezed-out toothpaste tube made of soft plastic",
    "a crumpled plastic wrapper from a chocolate bar",
    "a tangled bundle of plastic straws and stirrers",
    "a discarded plastic pen without ink",

    #  GLASS WASTE
    "a shattered transparent soda bottle with green labeling",
    "a broken wine bottle with sharp edges and label remnants",
    "a used glass jar with leftover jam inside and no lid",
    "a cracked window glass pane from a construction site",
    "a broken mirror with reflective shards on the floor",

    #  METAL WASTE
    "a rusty aluminum soda can with dents",
    "an open tin can with sharp jagged edges",
    "a bent steel rod from a demolished building",
    "a broken pair of scissors with exposed metal",
    "a discarded kitchen knife with a bent blade",

    #  PAPER WASTE
    "a crumpled notebook page with handwriting",
    "a greasy cardboard pizza box with cheese stains",
    "a pile of shredded office paper with colored ink",
    "a rolled-up newspaper with yellowed pages",
    "a torn cereal box made of paperboard",

    #  FOOD WASTE
    "banana peels with browning spots",
```

```
"half-eaten apple cores with teeth marks",  
"rotten tomatoes and squashed onions",  
"a stale slice of bread with mold spots",  
"leftover rice and curry on a used plate",  
"a pile of used tea bags and wet coffee grounds",  
"eggshells cracked open in halves",  
"chicken bones with some meat stuck to them",  
"fish heads and tails left over from cleaning",  
"spoiled milk in a discarded glass",  
"a used paper napkin soaked with soup",
```

TEXTILE WASTE

```
"a torn T-shirt with faded prints",  
"an old woolen sweater with holes",  
"a pair of broken shoes with detached soles",  
"a used bedsheet with stains and tears",  
"a discarded backpack with ripped zippers",  
"a bundle of worn-out socks and undergarments",
```

E-WASTE

```
"a cracked smartphone screen with exposed wires",  
"a burnt motherboard from a desktop computer",  
"used AA and AAA batteries in a pile",  
"an old CRT monitor covered in dust",  
"a broken USB cable with frayed ends",  
"a charger with a damaged plug",  
"a worn-out earphone set with one side missing",
```

MEDICAL WASTE (Use for visual-only datasets)

```
"a used syringe with a protective cap",  
"a discarded surgical mask with ear loops",  
"a blood-stained cotton bandage",  
"a broken thermometer with leaked mercury",  
"an empty plastic pill blister pack",  
"a used IV tube with saline residue",
```

HAZARDOUS WASTE

```
"an old car battery leaking acidic fluid",  
"a pesticide spray can with hazard symbols",  
"an empty paint can with dried paint on its rim",  
"a fluorescent tube light broken in half",  
"a used engine oil container with residue",
```

CERAMIC & CONSTRUCTION WASTE

```
"a cracked ceramic coffee mug",  
"a broken bathroom tile with jagged edges",  
"cement rubble from demolished walls",  
"a shattered toilet seat made of porcelain",  
"a dislodged piece of brick from a wall",
```

WOOD WASTE

```
"a broken wooden chair leg with splinters",  
"an old wooden photo frame with chipped edges",  
"a pile of sawdust and wood shavings",  
"a discarded plywood board with nails",  
"a snapped wooden broom handle",
```

MISCELLANEOUS WASTE

```
"a black garbage bag filled with mixed waste",  
"a crushed styrofoam food tray with food residue",
```

```

    "a used paper coffee cup with a plastic lid",
    "a torn party balloon lying on the ground",
    "a burnt-out matchstick next to a firecracker",
    "a broken plastic toy with missing limbs"
]

# Load CLIP model
model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32")
processor = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")

device = "cuda" if torch.cuda.is_available() else "cpu"
model.to(device)

# Classification smoothing over last 5 frames
prediction_queue = deque(maxlen=5)

# Start webcam
cap = cv2.VideoCapture(0)
if not cap.isOpened():
    print("Error: Webcam not accessible.")
    exit()

print("Press 'q' to quit...")

while True:
    ret, frame = cap.read()
    if not ret:
        print("Failed to grab frame.")
        break

    # Convert OpenCV BGR frame to RGB
    image_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    pil_image = Image.fromarray(image_rgb)

    # Process image and text
    inputs = processor(
        text=waste_classes,
        images=pil_image,
        return_tensors="pt",
        padding=True
    ).to(device)

    # Predict
    with torch.no_grad():
        outputs = model(**inputs)
        logits_per_image = outputs.logits_per_image # shape: [1, num_classes]
        probs = logits_per_image.softmax(dim=1)

    # Get highest probability label
    predicted_idx = probs.argmax().item()
    predicted_label = waste_classes[predicted_idx]
    prediction_queue.append(predicted_label)

    # Voting over last 5 predictions
    most_common_prediction = Counter(prediction_queue).most_common(1)[0][0]

    # Display Label
    display_text = f"Waste: {most_common_prediction}"
    cv2.putText(frame, display_text, (20, 40), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0

```

```

cv2.imshow("Waste Classification", frame)

# Quit on 'q'
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

# Release everything
cap.release()
cv2.destroyAllWindows()

```

Using a slow image processor as `use\_fast` is unset and a slow processor was saved with this model. `use\_fast=True` will be the default behavior in v4.52, even if the model was saved with a slow processor. This will result in minor differences in outputs. You'll still be able to use a slow processor with `use\_fast=False`. Press 'q' to quit...

KeyboardInterrupt

```

In [8]: import torch
        from transformers import CLIPProcessor, CLIPModel
        import cv2
        from PIL import Image
        import numpy as np
        from collections import deque, Counter

        # Waste class list (unchanged)
        waste_classes = [
            # ✅ PLASTIC WASTE
            "a crushed plastic water bottle with a blue cap and label",
            "a transparent plastic food container with leftover curry stains",
            "a disposable plastic spoon from a street food vendor",
            "a black plastic grocery bag caught in a tree",
            "a broken red plastic bucket with a missing handle",
            "a used plastic shampoo bottle with residue inside",
            "a squeezed-out toothpaste tube made of soft plastic",
            "a crumpled plastic wrapper from a chocolate bar",
            "a tangled bundle of plastic straws and stirrers",
            "a discarded plastic pen without ink",

            # ✅ GLASS WASTE
            "a shattered transparent soda bottle with green labeling",
            "a broken wine bottle with sharp edges and label remnants",
            "a used glass jar with leftover jam inside and no lid",
            "a cracked window glass pane from a construction site",
            "a broken mirror with reflective shards on the floor",

            # ✅ METAL WASTE
            "a rusty aluminum soda can with dents",
            "an open tin can with sharp jagged edges",
            "a bent steel rod from a demolished building",
            "a broken pair of scissors with exposed metal",
            "a discarded kitchen knife with a bent blade",

            # ✅ PAPER WASTE
            "a crumpled notebook page with handwriting",
            "a greasy cardboard pizza box with cheese stains",
            "a pile of shredded office paper with colored ink",
            "a rolled-up newspaper with yellowed pages",
            "a torn cereal box made of paperboard",

```

```
#  FOOD WASTE
"banana peels with browning spots",
"half-eaten apple cores with teeth marks",
"rotten tomatoes and squashed onions",
"a stale slice of bread with mold spots",
"leftover rice and curry on a used plate",
"a pile of used tea bags and wet coffee grounds",
"eggshells cracked open in halves",
"chicken bones with some meat stuck to them",
"fish heads and tails left over from cleaning",
"spoiled milk in a discarded glass",
"a used paper napkin soaked with soup",

#  TEXTILE WASTE
"a torn T-shirt with faded prints",
"an old woolen sweater with holes",
"a pair of broken shoes with detached soles",
"a used bedsheet with stains and tears",
"a discarded backpack with ripped zippers",
"a bundle of worn-out socks and undergarments",

#  E-WASTE
"a cracked smartphone screen with exposed wires",
"a burnt motherboard from a desktop computer",
"used AA and AAA batteries in a pile",
"an old CRT monitor covered in dust",
"a broken USB cable with frayed ends",
"a charger with a damaged plug",
"a worn-out earphone set with one side missing",

#  MEDICAL WASTE
"a used syringe with a protective cap",
"a discarded surgical mask with ear loops",
"a blood-stained cotton bandage",
"a broken thermometer with leaked mercury",
"an empty plastic pill blister pack",
"a used IV tube with saline residue",

#  HAZARDOUS WASTE
"an old car battery leaking acidic fluid",
"a pesticide spray can with hazard symbols",
"an empty paint can with dried paint on its rim",
"a fluorescent tube light broken in half",
"a used engine oil container with residue",

#  CERAMIC & CONSTRUCTION WASTE
"a cracked ceramic coffee mug",
"a broken bathroom tile with jagged edges",
"cement rubble from demolished walls",
"a shattered toilet seat made of porcelain",
"a dislodged piece of brick from a wall",

#  WOOD WASTE
"a broken wooden chair leg with splinters",
"an old wooden photo frame with chipped edges",
"a pile of sawdust and wood shavings",
"a discarded plywood board with nails",
"a snapped wooden broom handle",

#  MISCELLANEOUS WASTE
```

```

    "a black garbage bag filled with mixed waste",
    "a crushed styrofoam food tray with food residue",
    "a used paper coffee cup with a plastic lid",
    "a torn party balloon lying on the ground",
    "a burnt-out matchstick next to a firecracker",
    "a broken plastic toy with missing limbs"
]

# Mapping categories to bins
waste_category_map = {
    "plastic": "Blue Bin (Dry Waste)",
    "glass": "Blue Bin (Dry Waste)",
    "metal": "Blue Bin (Dry Waste)",
    "paper": "Blue Bin (Dry Waste)",
    "textile": "Blue Bin (Dry Waste)",
    "wood": "Blue Bin (Dry Waste)",
    "ceramic": "Blue Bin (Dry Waste)",
    "construction": "Blue Bin (Dry Waste)",
    "miscellaneous": "Blue Bin (Dry Waste)",
    "food": "Green Bin (Wet Waste)",
    "e-waste": "E-Waste Bin",
    "medical": "Red/Yellow Bin (Medical Waste)",
    "hazardous": "Black Bin (Hazardous Waste)"
}

def get_dustbin_type(predicted_label: str) -> str:
    label = predicted_label.lower()
    if any(word in label for word in ["plastic", "wrapper", "bucket", "bag", "pe
        return waste_category_map["plastic"]
    elif any(word in label for word in ["glass", "bottle", "jar", "mirror"]):
        return waste_category_map["glass"]
    elif any(word in label for word in ["metal", "can", "knife", "scissors", "ro
        return waste_category_map["metal"]
    elif any(word in label for word in ["paper", "cardboard", "newspaper", "page
        return waste_category_map["paper"]
    elif any(word in label for word in ["banana", "apple", "food", "bread", "ric
        return waste_category_map["food"]
    elif any(word in label for word in ["t-shirt", "sweater", "shoes", "bedsheet
        return waste_category_map["textile"]
    elif any(word in label for word in ["smartphone", "battery", "monitor", "usb
        return waste_category_map["e-waste"]
    elif any(word in label for word in ["syringe", "mask", "thermometer", "banda
        return waste_category_map["medical"]
    elif any(word in label for word in ["pesticide", "engine oil", "paint", "flu
        return waste_category_map["hazardous"]
    elif any(word in label for word in ["ceramic", "tile", "brick", "toilet", "c
        return waste_category_map["ceramic"]
    elif any(word in label for word in ["wood", "broom", "plywood", "sawdust"]):
        return waste_category_map["wood"]
    else:
        return "General Waste"

# Load CLIP model and processor
model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32")
processor = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")
device = "cuda" if torch.cuda.is_available() else "cpu"
model.to(device)

# Prediction buffer
prediction_queue = deque(maxlen=5)

```

```

# Webcam
cap = cv2.VideoCapture(0)
if not cap.isOpened():
    print("Error: Webcam not accessible.")
    exit()

print("Press 'q' to quit...")

while True:
    ret, frame = cap.read()
    if not ret:
        print("Failed to grab frame.")
        break

    image_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    pil_image = Image.fromarray(image_rgb)

    inputs = processor(
        text=waste_classes,
        images=pil_image,
        return_tensors="pt",
        padding=True
    ).to(device)

    with torch.no_grad():
        outputs = model(**inputs)
        logits_per_image = outputs.logits_per_image
        probs = logits_per_image.softmax(dim=1)

    predicted_idx = probs.argmax().item()
    predicted_label = waste_classes[predicted_idx]
    prediction_queue.append(predicted_label)

    most_common_prediction = Counter(prediction_queue).most_common(1)[0][0]
    dustbin = get_dustbin_type(most_common_prediction)

    # Display on webcam frame
    display_text = f"Waste: {most_common_prediction}"
    bin_text = f"Bin: {dustbin}"

    cv2.putText(frame, display_text, (20, 40), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0))
    cv2.putText(frame, bin_text, (20, 75), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 0, 0))
    cv2.imshow("Waste Classification", frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()

```

Press 'q' to quit...

```

In [ ]: import torch
        from transformers import CLIPProcessor, CLIPModel
        import cv2
        from PIL import Image
        import numpy as np
        from collections import deque, Counter

```



```

# Waste class descriptions
waste_classes = [
    # Plastic, Glass, Metal, etc. (same as your list, unchanged for brevity)
    "a crushed plastic water bottle with a blue cap and label",
    "a transparent plastic food container with leftover curry stains",
    "a disposable plastic spoon from a street food vendor",
    "a black plastic grocery bag caught in a tree",
    "a broken red plastic bucket with a missing handle",
    "a used plastic shampoo bottle with residue inside",
    "a squeezed-out toothpaste tube made of soft plastic",
    "a crumpled plastic wrapper from a chocolate bar",
    'waste notebooks',
    'book cover',
    'mobile cover',
    'benches to sit and chair',
    'spectacles',
    'iron bottles of different colors',
    'black board',
    'stationary miscellaneous',
    'damaged bluetooth earphones, wired earphones and accessories',
    'plastic box',
    "a tangled bundle of plastic straws and stirrers",
    "a discarded plastic pen without ink",
    "a shattered transparent soda bottle with green labeling",
    "a broken wine bottle with sharp edges and label remnants",
    "a used glass jar with leftover jam inside and no lid",
    "a cracked window glass pane from a construction site",
    "a broken mirror with reflective shards on the floor",
    "a rusty aluminum soda can with dents",
    "an open tin can with sharp jagged edges",
    "a bent steel rod from a demolished building",
    "a broken pair of scissors with exposed metal",
    "a discarded kitchen knife with a bent blade",
    "a crumpled notebook page with handwriting",
    "a greasy cardboard pizza box with cheese stains",
    "a pile of shredded office paper with colored ink",
    "a rolled-up newspaper with yellowed pages",
    "a torn cereal box made of paperboard",
    "banana peels with browning spots",
    "bag",
    "broken leather belt",
    'steel carrier',
    'id card and tag',
    "half-eaten apple cores with teeth marks",
    "rotten tomatoes and squashed onions",
    "a stale slice of bread with mold spots",
    "leftover rice and curry on a used plate",
    "a pile of used tea bags and wet coffee grounds",
    "eggshells cracked open in halves",
    "chicken bones with some meat stuck to them",
    "fish heads and tails left over from cleaning",
    "spoiled milk in a discarded glass",
    "a used paper napkin soaked with soup",
    "a torn T-shirt with faded prints",
    "an old woolen sweater with holes",
    "a pair of broken shoes with detached soles",
    "a used bedsheet with stains and tears",
    "a discarded backpack with ripped zippers",
    "a bundle of worn-out socks and undergarments",
    "a cracked smartphone screen with exposed wires",

```

```

    "a burnt motherboard from a desktop computer",
    "used AA and AAA batteries in a pile",
    "an old CRT monitor covered in dust",
    "a broken USB cable with frayed ends",
    "a wireless laptop mouse"
    "a charger with a damaged plug",
    "a worn-out earphone set with one side missing",
    "a used syringe with a protective cap",
    "a discarded surgical mask with ear loops",
    "a blood-stained cotton bandage",
    "a broken thermometer with leaked mercury",
    "an empty plastic pill blister pack",
    "a used IV tube with saline residue",
    "an old car battery leaking acidic fluid",
    "a pesticide spray can with hazard symbols",
    "an empty paint can with dried paint on its rim",
    "a fluorescent tube light broken in half",
    "a used engine oil container with residue",
    "a cracked ceramic coffee mug",
    "a broken bathroom tile with jagged edges",
    "cement rubble from demolished walls",
    "a shattered toilet seat made of porcelain",
    "a dislodged piece of brick from a wall",
    "a broken wooden chair leg with splinters",
    "an old wooden photo frame with chipped edges",
    "a pile of sawdust and wood shavings",
    "a discarded plywood board with nails",
    "a snapped wooden broom handle",
    "a black garbage bag filled with mixed waste",
    "a crushed styrofoam food tray with food residue",
    "a used paper coffee cup with a plastic lid",
    "a torn party balloon lying on the ground",
    "a burnt-out matchstick next to a firecracker",
    "a broken plastic toy with missing limbs"
]

# Dustbin mapping
waste_category_map = {
    "plastic": "Blue Bin (Dry Waste)",
    "glass": "Blue Bin (Dry Waste)",
    "metal": "Blue Bin (Dry Waste)",
    "paper": "Blue Bin (Dry Waste)",
    "textile": "Blue Bin (Dry Waste)",
    "wood": "Blue Bin (Dry Waste)",
    "ceramic": "Blue Bin (Dry Waste)",
    "construction": "Blue Bin (Dry Waste)",
    "miscellaneous": "Blue Bin (Dry Waste)",
    "food": "Green Bin (Wet Waste)",
    "e-waste": "E-Waste Bin",
    "medical": "Red/Yellow Bin (Medical Waste)",
    "hazardous": "Black Bin (Hazardous Waste)"
}

def get_dustbin_type(label: str) -> str:
    label = label.lower()
    if any(x in label for x in ["plastic", "bucket", "bag", "wrapper", "pen", "b
        return waste_category_map["plastic"]
    elif any(x in label for x in ["glass", "mirror", "jar"]):
        return waste_category_map["glass"]
    elif any(x in label for x in ["metal", "can", "knife", "scissors", "rod", "a

```

```

        return waste_category_map["metal"]
    elif any(x in label for x in ["paper", "cardboard", "box", "notebook", "news
        return waste_category_map["paper"]
    elif any(x in label for x in ["banana", "apple", "tomato", "bread", "rice",
        return waste_category_map["food"]
    elif any(x in label for x in ["t-shirt", "sweater", "shoes", "bedsheet", "sc
        return waste_category_map["textile"]
    elif any(x in label for x in ["smartphone", "monitor", "usb", "charger", "ba
        return waste_category_map["e-waste"]
    elif any(x in label for x in ["syringe", "mask", "bandage", "thermometer", "
        return waste_category_map["medical"]
    elif any(x in label for x in ["pesticide", "engine oil", "paint", "fluoresce
        return waste_category_map["hazardous"]
    elif any(x in label for x in ["ceramic", "tile", "toilet", "brick", "cement"
        return waste_category_map["ceramic"]
    elif any(x in label for x in ["wood", "broom", "plywood", "sawdust"]):
        return waste_category_map["wood"]
    else:
        return "General Waste"

# Load CLIP model and processor
device = "cuda" if torch.cuda.is_available() else "cpu"
model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32").to(device)
processor = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")

# Prediction buffer for smoothing
prediction_queue = deque(maxlen=5)

# Start webcam
cap = cv2.VideoCapture(0)
if not cap.isOpened():
    print("Error: Webcam not accessible.")
    exit()

print("Press 'q' to quit...")

while True:
    ret, frame = cap.read()
    if not ret:
        print("Failed to grab frame.")
        break

    image_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    pil_image = Image.fromarray(image_rgb)

    try:
        inputs = processor(
            text=waste_classes,
            images=pil_image,
            return_tensors="pt",
            padding=True
        ).to(device)

        with torch.no_grad():
            outputs = model(**inputs)
            logits_per_image = outputs.logits_per_image
            probs = logits_per_image.softmax(dim=1)

            predicted_idx = probs.argmax().item()
            predicted_label = waste_classes[predicted_idx]

```

```
prediction_queue.append(predicted_label)

# Most common label over last few frames
most_common_prediction = Counter(prediction_queue).most_common(1)[0][0]
dustbin = get_dustbin_type(most_common_prediction)

# Overlay info on video
cv2.putText(frame, f"Waste: {most_common_prediction}", (20, 40),
            cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
cv2.putText(frame, f"Bin: {dustbin}", (20, 75),
            cv2.FONT_HERSHEY_SIMPLEX, 0.65, (255, 255, 0), 2)

except Exception as e:
    cv2.putText(frame, f"Error: {e}", (20, 40),
                cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 0, 255), 2)

cv2.imshow("Waste Classification", frame)

if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()
```

Press 'q' to quit...

In []: